SPAI-GMRES-IR Project: Overall Guide and Code Flow

Read this first if you are completely new. The detailed sections below are useful only after this short foundation. Do not worry about memorizing every word on the first read.

Beginner foundation: computers, code, and math

What is this folder?

This folder is a software project: a collection of files that together tell a computer how to perform a task. The task is solving a large set of equations. Humans write the instructions as source code. A special program called a compiler translates that source code into an executable program that a computer can run.

The project contains several kinds of files:

File ending/name - Meaning

.cpp - C++ instructions mainly used for CPU work.

.cu - CUDA C++ instructions containing GPU-related work.

.hpp - A header: a shared description of functions and data. It is like a label on a box saying what the box contains and how to use it.

.md - A Markdown document: ordinary text with simple formatting.

.mtx - A Matrix Market text file containing a matrix.

CMakeLists.txt - Build instructions. It is not solver mathematics.

What are build, CMake, compiler, and test?

You will see these words often:

- Build: turn source code into an executable program.
- Compiler: the translator that turns C++ instructions into machine code.
- CMake: a helper that reads CMakeLists.txt and prepares build instructions for the compiler. Think of it as the assembly planner, not as part of the algorithm.
- Executable: a program you can run, such as cpu_fp64_ir.
- Run: start an executable with an input file.
- Test: a separate small program/script that deliberately checks expected behavior.

What is the mathematics in everyday terms?

The project solves A*x=b.

- A is a matrix, a table of numbers.
- x is a vector, a list of unknown numbers we want to find.
- b is a known vector.
- A*x means matrix-vector multiplication: each output number combines one row of A with the numbers in x.

The matrix is sparse, meaning most cells contain zero. Storing every zero would waste memory, so the code stores only nonzero values in a format called CSR. CSR is explained later.

The only three algorithm words to remember initially

- 1. SPAI builds a sparse helper matrix P that acts somewhat like an inverse of A.
- 2. GMRES uses P to find one improvement, called a correction d.
- 3. Iterative refinement adds that correction to the current answer and

repeats if necessary.

The simple story is:

```
read the sparse equation -> build helper P -> find correction d -> improve x
```

The later sections explain the terminology, code files, formulas, tests, and results in detail. When a term is unfamiliar, return to this foundation rather than trying to infer it from code.

A safe reading route

Do not try to learn this guide in one sitting. First read Sections 1, 7, and 8 to learn the project story and code route. Next read Sections 9, 10, and 11 until you can explain what SPAI, GMRES, and refinement each contribute. Read Sections 13 and 15 before making slides. Sections 12 and 14 are reference material for FP32, CUDA, and historical experiments.

Advanced words decoded before they appear

Word - Beginner meaning

Scaling - Resizing rows/numbers so their magnitudes are more balanced.

Transpose - Flip a matrix across its diagonal, turning rows into columns.

Least squares - When an exact fit is impossible, choose values with the smallest overall error.

QR - A stable fitting method used inside SPAI; it is a tool, not a fourth project stage.

Krylov basis - GMRES's growing set of useful trial directions.

Orthogonalize - Remove overlap so each direction adds new information.

Givens rotation - A small numerical step GMRES uses to update its internal mini-problem.

Diagnostic - Extra recorded information used to inspect a result or failure.

Purpose

This shared guide gives every team member the whole project story before they read their individual role brief. It answers: what problem is solved, what the active code implements, how a solver run travels through the source, and what the recorded results do and do not establish.

1. Project in one paragraph

The project solves sparse real square linear systems:



It uses a sparse approximate inverse (SPAI) preconditioner P, left-preconditioned unrestarted GMRES to compute solution corrections, and iterative refinement to improve the answer until the original system has small backward error. The active project supports CPU and CUDA GPU backends, each in uniform FP64 (double) and uniform FP32 (float). The CPU FP64 path is the reference path for understanding the implementation.

2. Scope: what is and is not implemented

The literature plan and project name mention BSPAI and mixed precision. The active code does not implement bucketed SPAI or a mixed-precision solver.

Use this statement in a presentation:

We implemented and validated uniform-precision SPAI-GMRES-IR variants in FP64 and FP32 for CPU and CUDA. BSPAI and mixed precision are literature-motivated future directions, not active implementation claims.

Do not claim a full reproduction of published BSPAI/mixed-precision tables or a performance speedup without a specific documented benchmark.

3. Why use this kind of solver?

Many scientific matrices are sparse: most entries are zero. A dense ntimes n matrix stores n^2 values, but sparse storage keeps only nonzeros. A matrix inverse is often dense even if the original matrix is sparse, so forming A^{-1} directly is usually impractical.

The project combines four ideas:

- CSR storage stores only nonzero values.
- SPAI builds a sparse approximation Papprox A^{-1}.
- GMRES is a Krylov iterative method that mainly needs matrix-vector products.
- Iterative refinement recomputes the original residual and repeatedly improves the full solution.

4. Repository map

Location - Purpose

README.md - Build, run, submission, and archive instructions.

CMakeLists.txt - Root build graph for CPU, optional CUDA, programs, and tests.

include/bspai/ - Public C++ declarations and result structures.

src/ - Production implementations.

tests/ - Regression tests, small fixtures, numerical checkers, package tests.

docs/ - Algorithm contracts, validation records, and cleanup evidence.

presentation/team_guides/ - Team learning material and PDF sources.

results/ - Pointer to preserved historical results.

archives/research_20261019.zip - Historical data, traces, profiles, campaigns, reports, and provenance.

submission/ - Source archive intended for submission.

5. Solver variants and command line

Program - Backend - Arithmetic

cpu_fp64_ir - CPU - FP64 / double

cpu_fp32_ir - CPU - FP32 / float

gpu_fp64_ir - CUDA GPU - FP64 / double

gpu_fp32_ir - CUDA GPU - FP32 / float

All use:

```
MATRIX.mtx [epsilon=0.3] [outercap=20] [innercap=0]
```

- MATRIX.mtx: sparse Matrix Market input.
- epsilon: SPAI construction target.
- outercap: at most 20 refinement corrections.
- innercap: at most this many GMRES iterations; 0 means n, the matrix size.

The command-line program creates the right-hand side internally:

```
b_i=1/sqrt n,
```

so it has equal components and unit 2-norm.

6. CPU FP64 source map

File - Responsibility

src/solver_main.cpp - Parses arguments/input, creates RHS and options, invokes a solver, prints output.

src/cpu.cpp - Matrix Market parsing, CSR construction, sparse multiply, and transpose.

include/bspai/cpu.hpp - Declares the CsrMatrix type.

src/spai.cpp - Constructs the FP64 sparse approximate inverse.

include/bspai/spai.hpp - SPAI options, fit records, statuses, and result type.

src/gmres.cpp - FP64 left-preconditioned GMRES.

include/bspai/gmres.hpp - GMRES options, histories, statuses, and result type.

src/ir.cpp - FP64 iterative refinement and original-system diagnostics.

include/bspai/ir.hpp - Iterative-refinement options, histories, diagnostics, result type.

src/main.cpp - Matrix-probe diagnostic program, not the main solver.

7. Input: Matrix Market and CSR

Matrix Market input

The parser accepts coordinate Matrix Market input, where each nonzero is:

```
row column value
```

For example:

```
%%MatrixMarket matrix coordinate real general
3 3 4
1 1 10
1 3 2
2 2 5
3 1 -1
```

represents:

```
A=begin{bmatrix}10&0&20&5&0-1&0&0end{bmatrix}.
```

Matrix Market uses one-based indices; the C++ code converts them to zero-based indices. It validates dimensions, supported format, indices, and finite values.

CSR storage

The example matrix is stored in compressed sparse row form as:

```
values         = [10, 2, 5, -1]
column_indices = [ 0, 2, 1,  0]
row_offsets    = [ 0, 2, 3,  4]
```

Entries for row i lie between row_offsets[i] and row_offsets[i+1]-1. The sparse multiplication is:

$$(Ax)_i = \sum_{j \text{ stored in row } i} a_{ij}x_j.$$

The CSR constructor sorts entries, sums duplicate locations in stable input order, and removes resulting zeros. The multiply function is used throughout the solver.

8. Full CPU FP64 execution flow

1. Start cpu_fp64_ir.
2. solver_main.cpp reads command-line parameters.
3. cpu.cpp parses the Matrix Market file into CSR matrix A.
4. solver_main.cpp creates b and the default options.
5. ir.cpp calls construct_spai(A).
6. spai.cpp builds sparse preconditioner P.
7. ir.cpp forms x0 = P*b.
8. ir.cpp forms original residual r = b - A*x.
9. gmres.cpp finds correction d by approximately solving P*A*d = P*r.
10. ir.cpp updates x = x + d.
11. ir.cpp recomputes original-system diagnostics.
12. Return success, named failure, or iteration-limit status.
13. solver_main.cpp prints status, backward error, and solution values.

The same information as a call graph:

```
main [solver_main.cpp]
  |
  +-- read_matrix_market() [cpu.cpp]
  |
  +-- spai_gmres_ir(A,b,options) [ir.cpp]
        |
        +-- construct_spai(A,options) [spai.cpp]
        |
        +-- refine_with_preconditioner(A,P,b,options) [ir.cpp]
              |
              +-- left_preconditioned_gmres(A,P,r,options) [gmres.cpp]
                    |
                    +-- apply_left_preconditioned(v) = P(A(v))
                          |
                          +-- CsrMatrix::multiply() [cpu.cpp]
```

9. SPAI construction

The preconditioner makes the system easier for GMRES:

$$P \approx A^{-1}, \qquad PA \approx I.$$

The code first computes row scaling:

$$d_k = 1/\max_j |a_{kj}|, \qquad D = \mathrm{diag}(d_k),$$

and forms:

$$B = A^T D.$$

It then constructs one sparse column at a time for a right approximate inverse N. For column k, it starts with the pattern J={k}, identifies the touched shadow rows I, and solves:

$$\min_m \|B(I,J)m - e_k(I)\|_2.$$

The code uses Householder QR. If the residual is too large, it gathers candidate columns, scores them, adds at most beta=8 candidates, and refits. It permits at most alpha=lceil n/8rceil growth/refit events. The default target is epsilon=0.3.

Finally it assembles:

$$P = N^T D.$$

This is a sparse approximation, never a dense exact inverse.

10. GMRES correction solve

At any outer step the current residual is:

$$r = b - Ax.$$

GMRES solves for correction d:

$$PAd = Pr.$$

It does not build the matrix PA. Its operator is applied as:

$$v \mapsto Av \mapsto P(Av).$$

Starting from a zero correction, GMRES normalizes the preconditioned residual and builds a Krylov basis through Arnoldi iteration. Each iteration:

- 1. computes w=PAv_j;
- 2. orthogonalizes w against the existing basis with modified Gram-Schmidt;
- 3. records an upper-Hessenberg coefficient column;
- 4. applies Givens rotations to the small least-squares problem;
- 5. back-substitutes to produce a candidate d;
- 6. recomputes the actual residual before accepting it.

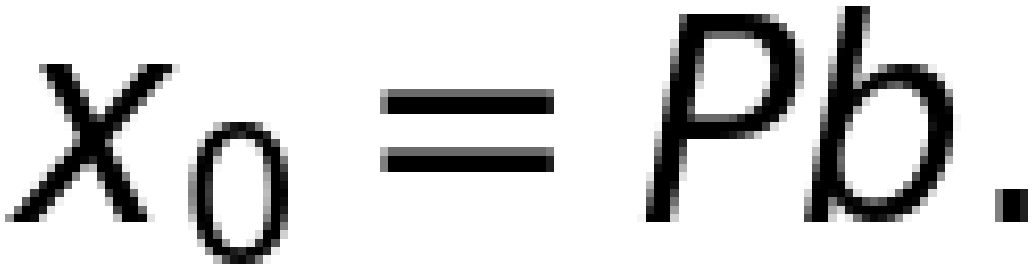
For FP64, the GMRES criterion is:

```
frac{|Pr-PAd|_2}{|Pr|_2}<=10^{-8}.
```

GMRES is unrestarted and can use up to n iterations by default. It has explicit error statuses for invalid input, a zero preconditioned RHS, Arnoldi breakdown, iteration limit, nonconvergence, and nonfinite arithmetic.

11. Iterative refinement

The outer loop begins with:



Then it repeats at most 20 times:

```
r = b - A*x
d = GMRES(A, P, r, zero initial correction)
if GMRES fails: stop with inner_failure
x = x + d
recompute residual and backward error
if backward error passes: success
if x did not change: stagnation
```

The final acceptance criterion is based on the original problem:

```
eta(x)=frac{|b-Ax|_infinity} {|A|_infinity|x|_infinity+|b|_infinity} <=10*2^{-53}.
```

The distinction matters: GMRES checks a preconditioned correction residual, while iterative refinement accepts the final answer using the original-system backward error.

12. FP32 and CUDA variants

FP32 has a separate CPU implementation in src/fp32_cpu.cpp and include/bspai/fp32.hpp. It is not a simple type substitution because FP32 has far lower precision and a narrower range. The code therefore has separate matrix types, range checks, statuses, and tolerances. FP32 GMRES uses 1e-4 and its outer target is 10*2^{-24}.

CUDA work is split by subsystem:

```
cuda_spmv.cu          GPU sparse matrix-vector products
cuda_spai.cu          GPU FP64 SPAI construction
cuda_gmres.cu         GPU FP64 GMRES
cuda_ir.cpp           GPU FP64 integration; outer refinement is CPU-staged
fp32_cuda_spmv.cu     GPU FP32 sparse products
fp32_cuda_spai.cu     GPU FP32 SPAI
fp32_cuda_gmres.cu    GPU FP32 GMRES
```

CPU/GPU results are not required to be bit-for-bit equal because floating-point operation order can differ. The project validates defined numerical properties instead.

13. Testing and current results

Tests cover CSR, SPAI, GMRES, iterative refinement, FP32, CUDA components, CLI behavior, fixed numerical snapshots, and submission/archive utilities. Independent Python checkers reconstruct numerical quantities rather than only trusting the solver's own report.

The recorded final validation state is:

Check	Recorded outcome
CPU regression suites	5/5 pass

CUDA regression suites - 8/8 pass

CLI checks across all four programs - 32/32 pass

CPU fixed snapshots - 16 records, 500 Decimal80 checks pass

CUDA Decimal80-oriented checks - zero recorded failures in listed checks

Small deterministic-fixture backward errors were about 10^{-8} in FP32 and 10^{-17} in FP64. These are correctness results, not benchmark speedup claims and not guarantees for every sparse matrix.

14. Historical archive

Older matrices, raw experiment traces, profiles, campaign tools, reports, and provenance are preserved in:

```
archives/research_20261019.zip
```

Verify or restore it with:

```
python scripts/research_archive.py verify
python scripts/research_archive.py restore
```

The archive is historical evidence. Do not describe its timing data as freshly rerun unless you show the precise environment, matrix, parameters, measurement method, and date.

15. Safe presentation narrative

"Our project solves sparse linear systems using a sparse approximate inverse preconditioner, GMRES, and iterative refinement. The CPU reference path stores matrices in CSR format, builds an adaptive sparse preconditioner using QR-based least-squares fits, and computes refinement corrections using left-preconditioned unrestarted GMRES. We accept the final solution only when the original system meets a backward-error criterion. The project supports uniform FP64 and FP32 CPU/GPU variants. Our recorded evidence demonstrates correctness on controlled deterministic tests; it does not claim a completed mixed-precision/BSPAI implementation or an unqualified performance speedup."

16. Recommended presentation sequence

- 1. State the sparse system Ax=b and why sparse methods matter.
- 2. Show the SPAI-GMRES-IR pipeline.
- 3. Explain Matrix Market input and CSR.
- 4. Explain SPAI construction at a high level.
- 5. Explain GMRES corrections and iterative refinement.
- 6. Explain FP32/CUDA variants and their limitations.
- 7. Present validation outcomes and their scope.
- 8. Conclude with limitations and future directions.

17. Questions everyone should handle

What does SPAI do? It constructs sparse Papprox A^{-1}, so PA is easier for GMRES.

Why GMRES? It is suitable for nonsymmetric sparse systems and relies on matrix-vector products instead of a dense factorization.

Why iterative refinement? It uses the original residual to compute and apply repeated corrections to the full solution.

Why CSR? It stores only nonzeros and makes sparse multiplication efficient.

How is success measured? By original-system normwise backward error.

What is not implemented? Active BSPAI bucketing and mixed precision.

Related team guides

18. Current implementation: the complete call map

This section connects the explanation above to the code that is in the repository now. A call means that one

function asks another function to do a smaller job. Read each arrow as "then calls."

CPU FP64 executable

```
solver_main.cpp
  -> read_matrix_market()             [cpu.cpp]
  -> build the known vector x_true
  -> csr_spmv(A, x_true) to form b    [cpu.cpp]
  -> cpu_spai_gmres_ir()              [ir.cpp]
       -> construct_spai()            [spai.cpp]
       -> apply P to b for x_0
       -> repeat outer refinement steps:
            residual r = b - A*x      [cpu.cpp]
            gmres_left_preconditioned()[gmres.cpp]
            update x = x + d
            calculate backward error
  -> independently recompute final quality
  -> print status, counts, timing, and solution
```

The FP32 CPU program has the same mathematical stages, but uses the separately implemented fp32_... files. It is separate because changing precision affects constants, stopping thresholds, intermediate rounding, library calls, and validation-not only the spelling of the scalar type.

Native CUDA FP64 executable

```
solver_main.cpp
  -> read A on the CPU
  -> cuda_gpu_spai_gmres_ir()              [cuda_ir.cpp]
       -> cuda_construct_spai()            [cuda_spai.cu]
            GPU scaling, reduced systems, QR solves, residual scoring,
            and sparse P assembly
       -> cuda_refine_with_preconditioner() [cuda_ir.cpp]
            CPU forms outer residual and checks outer accuracy
            CudaGmresContext                [cuda_gmres.cu]
              retains A and P on GPU
              performs GPU SpMV and vector operations
              returns a correction d
```

The CUDA FP32 route follows the same division of responsibility through fp32_cuda_spai.cu, fp32_cuda_ir.cpp, and fp32_cuda_gmres.cu.

Important naming distinction

- cuda_spai_gmres_ir is a diagnostic hybrid path: it can construct SPAI on the CPU and use CUDA for GMRES.
- cuda_gpu_spai_gmres_ir is the native GPU-SPAI path used to study the CUDA implementation.

These names are similar, so pointing out this distinction during a demonstration prevents an accidental claim that a CPU-built preconditioner was built on the GPU.

19. What data lives where in the CUDA solver?

The input Matrix Market file is parsed on the CPU into CSR arrays. During GPU SPAI construction, the matrix is uploaded and GPU kernels perform scaling, reduced-matrix gathering, fitted-residual calculation, candidate scoring, and final sparse assembly. The host still manages irregular control decisions such as pattern sets and candidate selection.

After SPAI construction, the canonical sparse preconditioner P is available on the host. The persistent GMRES context uploads A and P once and retains them across refinement corrections. Therefore, the entire matrix is not copied to and from the GPU at every GMRES iteration. Small scalar observations needed for Arnoldi and the small Hessenberg problem cross the boundary; right-hand sides and final correction vectors also cross at solve boundaries.

For FP32 CUDA GMRES, basis storage is allocated lazily in groups of 16 vectors. This matters because unrestarted GMRES may need many basis vectors, but easy systems may converge after very few. The old eager scheme could reserve about 576 MB for a size-12,000 one-step identity problem; the slab scheme initially needs only one 768 KB device slab plus about 1 KB for the small host projected system.

20. Three levels of correctness checking

The implementation does not trust a single number.

- 1. SPAI construction evidence: each approximate-inverse column records its pattern-growth and fitted-residual behavior.
- 2. GMRES correction verification: after GMRES proposes d, the solver explicitly recomputes how well it satisfies the preconditioned correction equation. The acceptance ratio is

$$\frac{\|Pr - PAd\|_2}{\|Pr\|_2}.$$

The present acceptance bound is 1e-8 for FP64 and 1e-4 for FP32.

- 3. Whole-solution verification: the CPU independently evaluates the normwise backward error using the original A, b, and returned x:

$$\eta = \frac{\|b - Ax\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty}.$$

The project checks approximately 10 * 2^-53 for FP64 and 10 * 2^-24 for FP32. "Backward error" asks whether the computed answer exactly solves a nearby problem; it is scale-aware and is more informative than quoting only the raw residual.

21. Current results in plain language

The strongest current correctness evidence is that the active CPU suite passes 5 of 5 CTest cases and the active CUDA suite passes 8 of 8. Four command-line solver variants passed 32 Decimal80 reference checks. "Decimal80" means a high-precision decimal calculation is used as an independent numerical reference rather than trusting the same arithmetic as the implementation under test.

The latest CUDA engineering measurements are narrower:

- Persistent FP32 GMRES context: an isolated fixed-preconditioner refinement benchmark fell from roughly 10.50-10.85 ms to 9.71-9.80 ms, about 7-10%. The complete solver remained about 2.15-2.16 s, so this did not establish a visible end-to-end improvement.
- Lazy FP32 GMRES storage: a size-12,000 one-step identity case fell from 220.103 ms to 1.331 ms, and an exact-initial-solution case from 16.634 ms to 0.907 ms. On one stencil_8192 workload, pooled medians were 2639.473 ms before and 2290.805 ms after, a 13.21% reduction or about 1.15x speedup.
- Fused GPU SPAI residual work: an exploratory stencil constructor median was about 2153 ms versus 2282-2307 ms previously, about 5-7% lower. A row-parallel candidate was 5-7% slower and was rejected.

These are measurements on particular matrices and one GPU environment. They demonstrate that the optimizations were useful on those cases; they are not evidence of a universal speedup.

22. Present scope and honest limitations

The project currently implements SPAI-GMRES-IR in four uniform-precision variants: CPU FP64, CPU FP32, CUDA FP64, and CUDA FP32. The name BSPAI describes the central research direction and literature baseline, but a block-SPAI implementation and a mixed-precision solver are not yet production features. Say this explicitly.

Other limitations are that CPU and GPU SPAI can make different but valid choices when candidate scores tie; singular QR systems can map to different low-level status codes; FP32 input duplicate summation keeps the parser's original ordering; and unrestarted GMRES can still consume substantial memory when many iterations are genuinely required.

Finally, the current submission/SPAI_GMRES_IR_Submission.zip does not match the previously recorded final validated archive: the current file has 64 members and a size of 147,400 bytes, while the recorded validated package had 99 files and 313,131 bytes. It must be regenerated and independently verified before anyone calls it the final submitted package.

23. Top-to-bottom file flow for a complete run

This is the complete route a professor can follow from the command line to the reported answer.

- 1. Build selection: CMakeLists.txt and cmake/CpuTargets.cmake or cmake/CudaTargets.cmake decide which source files and CUDA libraries form the requested executable.
- 2. Executable entry: src/solver_main.cpp::main(argc, argv) receives the matrix path, SPAI tolerance, outer cap, and inner cap. It returns an operating-system exit code: zero for an accepted solver result, nonzero for invalid arguments, file errors, or solver failure.
- 3. Input: read_matrix_market(std::istream&) in src/cpu.cpp returns a canonical CsrMatrix. For FP32, solver_matrix(...) checks range and converts it to Fp32CsrMatrix.
- 4. Problem creation: the driver creates rhs, configures IrOptions/Fp32IrOptions, and dispatches through the build-selected solver macro.
- 5. Construction: CPU variants call construct_spai; native CUDA variants call cuda_construct_spai or fp32_cuda_construct_spai. The output contains P, statuses, per-column histories, and-on CUDA-allocation, transfer, stream, and timing telemetry.
- 6. Refinement: refine_with_preconditioner, cuda_refine_with_preconditioner, or fp32_cuda_refine_with_preconditioner forms residuals, calls the correct GMRES implementation, applies corrections, and records every outer step.
- 7. Inner solve: the relevant GMRES function returns a status, correction vector, iteration count,

true/estimated residuals, and history. CUDA wrappers also report transfers.

- 8. Output: the driver prints solver identity, status, convergence counts, backward error, construction information, elapsed time, and serialized solution values.

24. Public API map: inputs and outputs

Function - Important inputs - Output

read_matrix_market(input) - text stream - canonical CsrMatrix

construct_spai(A, options) - matrix; epsilon, alpha, beta - SpaiResult with scaling, transformed matrix, inverse, P, columns and status

left_preconditioned_gmres(A,P,rhs,options,initial) - two matrices, correction RHS, tolerance, cap, optional start - GmresResult with correction, status, histories and residuals

refine_with_preconditioner(A,P,rhs,options,reference) - fixed preconditioner and IR/GMRES options - IrResult with final solution and nested histories

spai_gmres_ir(A,rhs,options,reference) - complete CPU problem - IrResult including SPAI construction

cuda_construct_spai(A,options,device) - FP64 matrix/options/GPU number - CudaSpaiResult = numerical construction + telemetry

cuda_gpu_spai_gmres_ir(A,rhs,options,reference,device) - complete native CUDA FP64 problem - CudaIrResult = ordinary IR result + CUDA transfers/telemetry

fp32_cuda_construct_spai(A,options,device) - FP32 matrix/options/GPU number - Fp32CudaSpaiResult

fp32_cuda_spai_gmres_ir(A,rhs,options,device) - complete native CUDA FP32 problem - Fp32IrResult containing the CUDA-built preconditioner result

epsilon is the SPAI fit tolerance. alpha is the maximum number of adaptive growth steps. beta bounds how many candidates are selected at one growth. The GMRES tolerance controls the correction target; its cap bounds the Krylov basis. The outer cap bounds refinement corrections. device=0 selects the first CUDA GPU unless another index is supplied.

25. Return structures are the audit trail

SpaiResult contains overall status/message, scaling, the scaled transpose, approximate inverse, preconditioner, and every SpaiColumn. A column stores patterns, coefficients, fit/growth counts, residuals, pivot information, and full SpaiFit history.

GmresResult contains status/message, convergence flag, iteration count, correction, initial/final residual measures, basis and Hessenberg data, and one GmresIteration per step. Each iteration records its estimate, recomputed check, original residual, Arnoldi subdiagonal, breakdown/verification/acceptance flags, and candidate solution.

IrResult nests both: final status/solution, outer-step count, construction, per-step diagnostics, every inner GMRES result, correction right-hand sides, and whether each update was applied. CudaIrResult adds context constructions, matrix uploads, transfer counters, and CUDA SPAI telemetry. This is why validation can inspect how an answer was produced rather than only seeing the last vector.

- 01_algorithm_and_research.md: Member 1's theory and research guide.
- 02_cpu_fp64_implementation.md: Member 2's CPU source guide.
- 03_fp32_and_cuda_variants.md: Member 3's variant guide.
- 04_validation_results_and_delivery.md: Member 4's validation guide.